Lab Assignment- 7

Ridhika Bansal, 1024150280
2O31

Topics:
A. Polymorphism
B. Virtual Function and Pure Virtual Function
C. Operator Overloading

1. Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

```
#include <iostream>
using namespace std;

class ShapeBase {
protected:
    int l, b;

public:
    void setDimensions(int x, int y) {
        l = x;
        b = y;
    }

    virtual float getArea() {
        return 0;
    }
};

class Rect : public ShapeBase {
public:
    float getArea() {
        return l * b;
    }
};

class Tri : public ShapeBase {
public:
    float getArea() {
        return 0.5 * l * b;
```

Output

```
Rectangle Area = 32
Triangle Area = 16


=== Code Execution Successful ===
```

```
    }
};

int main() {
    ShapeBase *ptr;

    Rect obj1;
    Tri obj2;

    ptr = &obj1;
    ptr->setDimensions(8, 4);
    cout << "Rectangle Area = " << ptr->getArea() << endl;

    ptr = &obj2;
    ptr->setDimensions(8, 4);
    cout << "Triangle Area = " << ptr->getArea() << endl;

    return 0;
}
```

2. Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

```
#include <iostream>
using namespace std;

class Figure {
public:
    virtual void calcArea() = 0;
    virtual void info() = 0;
};

class CircleShape : public Figure {
    float r;
public:
    CircleShape(float x) { r = x; }

    void calcArea() {
        cout << "Circle Area: " << 3.1416 * r * r << endl;
    }

    void info() {
        cout << "Type: Circle | Radius: " << r << endl;
```

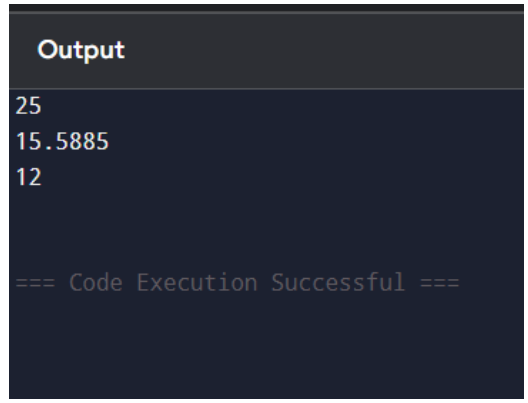
```
    }
};

class RectShape : public Figure {
    float len, wid;
public:
    RectShape(float a, float b) { len = a; wid = b; }

    void calcArea() {
        cout << "Rectangle Area: " << len * wid << endl;
    }

    void info() {
        cout << "Type: Rectangle | L=" << len << ", W=" << wid << endl;
    }
};

int main() {
    Figure *f;

    CircleShape c(5);
    RectShape r(6, 3);

    f = &c;
    f->info();
    f->calcArea();

    f = &r;
    f->info();
    f->calcArea();

    return 0;
}
```

```
Output
Type: Circle | Radius: 5
Circle Area: 78.54
Type: Rectangle | L=6, W=3
Rectangle Area: 18


=== Code Execution Successful ===
```

3. Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

```
#include <iostream>
#include <cmath>
using namespace std;

class TriArea {
public:
    float calc(float base, float height) {
        return 0.5 * base * height;
    }
```

```
    float calc(float side) {
        return (sqrt(3) / 4) * side * side;
    }

    float calc(float a, float b, float c) {
        float s = (a + b + c) / 2;
        return sqrt(s * (s - a) * (s - b) * (s - c));
    }
};

int main() {
    TriArea t;
    cout << t.calc(10, 5) << endl;
    cout << t.calc(6) << endl;
    cout << t.calc(5, 5, 8) << endl;
    return 0;
}
```



4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

```
#include <iostream>
using namespace std;

class Student {
protected:
    string name;
    int roll;

public:
    Student(string n, int r) {
        name = n;
        roll = r;
    }

    virtual void display() = 0;
};

class Engineering : public Student {
    string branch;

public:
    Engineering(string n, int r, string b) : Student(n, r) {
```

```
        branch = b;
    }

    void display() {
        cout << "Engineering Student -> Name: " << name
            << ", Roll: " << roll
            << ", Branch: " << branch << endl;
    }
};

class Medicine : public Student {
    string specialization;

public:
    Medicine(string n, int r, string s) : Student(n, r) {
        specialization = s;
    }

    void display() {
        cout << "Medicine Student -> Name: " << name
            << ", Roll: " << roll
            << ", Specialization: " << specialization << endl;
    }
};

class Science : public Student {
    string subject;

public:
    Science(string n, int r, string sub) : Student(n, r) {
        subject = sub;
    }

    void display() {
        cout << "Science Student -> Name: " << name
            << ", Roll: " << roll
            << ", Subject: " << subject << endl;
    }
};

int main() {
    Student* s[3];

    Engineering e("Ridhika", 280, "Artificial Intelligence");
    Medicine m("Aman", 102, "Cardiology");
    Science sc("Riya", 103, "Physics");
```

```
    s[0] = &e;
    s[1] = &m;
    s[2] = &sc;

    for (int i = 0; i < 3; i++) {
        s[i]->display();
    }

    return 0;
}
```

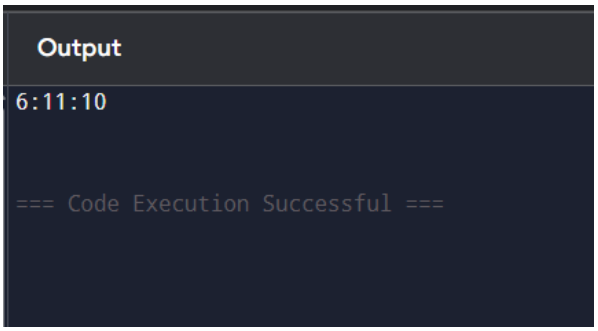
```
Output                                                    Clear
Engineering Student -> Name: Ridhika, Roll: 280, Branch: Artificial Intelligence
Medicine Student -> Name: Aman, Roll: 102, Specialization: Cardiology
Science Student -> Name: Riya, Roll: 103, Subject: Physics


=== Code Execution Successful ===
```

5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.

```
int main(){
Time t1(5,15,34),t2(9,53,58),t3;
t3 = t1 + t2;
t3.show();
}
```

```
#include <iostream>
using namespace std;

class ClockTime {
    int hr, min, sec;

public:
    ClockTime(int h = 0, int m = 0, int s = 0) {
        hr = h; min = m; sec = s;
    }

    ClockTime operator+(ClockTime t) {
        ClockTime res;

        res.sec = sec + t.sec;
```

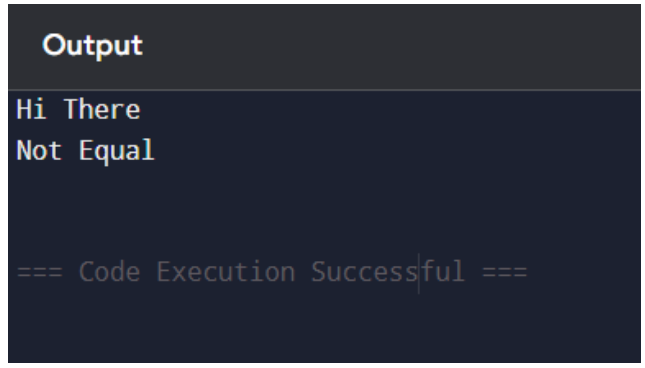
```
        res.min = min + t.min + res.sec / 60;
        res.sec %= 60;

        res.hr = hr + t.hr + res.min / 60;
        res.min %= 60;

        return res;
    }

    void print() {
        cout << hr << ":" << min << ":" << sec << endl;
    }
};

int main() {
    ClockTime t1(2, 50, 40), t2(3, 20, 30);

    ClockTime t3 = t1 + t2;
    t3.print();

    return 0;
}
```



6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

```
#include <iostream>
using namespace std;

class MyString {
    string data;

public:
    MyString(string s = "") {
        data = s;
    }

    bool operator==(MyString s) {
        return data == s.data;
    }

    MyString operator+(MyString s) {
        return MyString(data + s.data);
    }

    void show() {
```

```
        cout << data << endl;
    }
};

int main() {
    MyString a("Hi "), b("There");

    MyString c = a + b;
    c.show();

    if (a == b)
        cout << "Equal\n";
    else
        cout << "Not Equal\n";

    return 0;
}
```



7. Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

```
#include <iostream>
#include <vector>
using namespace std;

class Mat {
    vector<vector<int>> mat;
    int r, c;

public:
    Mat(int rows = 0, int cols = 0) {
        r = rows;
        c = cols;
        mat.resize(r, vector<int>(c));
    }

    void read() {
        cout << "Enter elements (" << r << " x " << c << "):\n";
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++)
                cin >> mat[i][j];
    }

    void print() const {
        cout << "Result matrix:\n";
        for (int i = 0; i < r; i++) {
```

```
            for (int j = 0; j < c; j++)
                cout << mat[i][j] << " ";
            cout << endl;
        }
    }

    friend Mat operator*(const Mat &a, const Mat &b);
};

Mat operator*(const Mat &a, const Mat &b) {
    if (a.c != b.r) {
        cout << "Matrix multiplication not possible!\n";
        return Mat();
    }

    Mat res(a.r, b.c);

    for (int i = 0; i < a.r; i++) {
        for (int j = 0; j < b.c; j++) {
            res.mat[i][j] = 0;
            for (int k = 0; k < a.c; k++) {
                res.mat[i][j] += a.mat[i][k] * b.mat[k][j];
            }
        }
    }
    return res;
}

int main() {
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    Mat m1(r1, c1), m2(r2, c2);

    m1.read();
    m2.read();

    Mat m3 = m1 * m2;

    if (r1 > 0 && c2 > 0 && c1 == r2)
        m3.print();
```

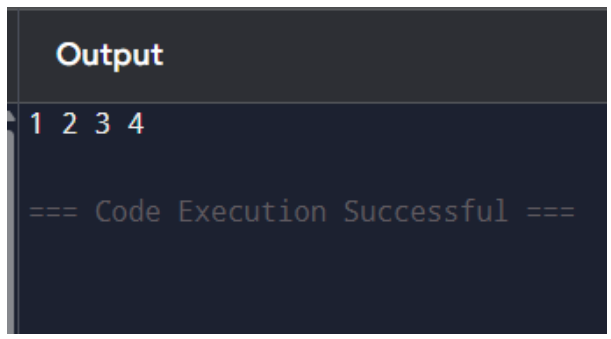
Output

```
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements (2 x 2):
1 2
3 4
Enter elements (2 x 2):
5 6
7 8
Result matrix:
19 22
43 50


=== Code Execution Successful ===
```

```
    return 0;
}
```

8. Overload '[]' to check array index out of bounds problem at run time.

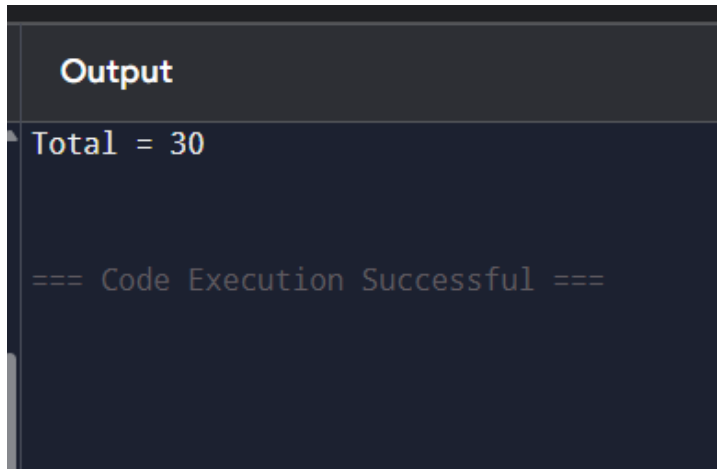
```
#include <iostream>
using namespace std;

class SafeArray {
    int arr[50];
    int n;

public:
    SafeArray(int size) {
        n = size;
    }

    int& operator[](int idx) {
        if (idx < 0 || idx >= n) {
            cout << "Invalid Index!" << endl;
            exit(0);
        }
        return arr[idx];
    }
};

int main() {
    SafeArray a(4);

    for (int i = 0; i < 4; i++)
        a[i] = i + 1;

    for (int i = 0; i < 4; i++)
        cout << a[i] << " ";

    return 0;
}
```



9. Overload '()' to input arbitrary number of input arguments for an object.

```
#include <iostream>
#include <cstdarg>
using namespace std;

class Adder {
    int total;
```

```
public:
    Adder() { total = 0; }

    void operator()(int count, ...) {
        va_list list;
        va_start(list, count);

        total = 0;
        for (int i = 0; i < count; i++) {
            total += va_arg(list, int);
        }

        va_end(list);
    }

    void show() {
        cout << "Total = " << total << endl;
    }
};

int main() {
    Adder a;

    a(3, 5, 10, 15);
    a.show();

    return 0;
}
```



10. Write a program in C++ to overload input operator (>>) and output operator (<<).

```
#include <iostream>
#include <string>
using namespace std;

class Stu {
    int id;
    string nm;

public:
    friend istream& operator>>(istream &in, Stu &s);
    friend ostream& operator<<(ostream &out, const Stu &s);
};
```

```
istream& operator>>(istream &in, Stu &s) {
    cout << "Enter ID: ";
    in >> s.id;

    cout << "Enter Name: ";
    in.ignore();            // clear newline from buffer
    getline(in, s.nm);      // allows full name with spaces

    return in;
}

ostream& operator<<(ostream &out, const Stu &s) {
    out << "ID: " << s.id << "\nName: " << s.nm << endl;
    return out;
}

int main() {
    Stu s;
    cin >> s;
    cout << s;

    return 0;
}
```

Output

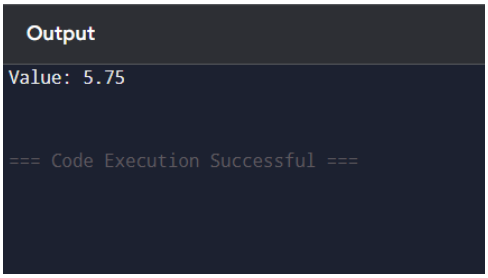
```
Enter ID: 280
Enter Name: Ridhika
ID: 280
Name: Ridhika


=== Code Execution Successful ===
```

11. Write a program to convert basic data type (float) to user defined data type (object).

```
class Test {
private:
    //…
public:
    Test(data_type) {
        // conversion code
    }
};
```

```
#include <iostream>
using namespace std;

class Test {
    float value;

public:
    Test(float x) {
        value = x;
```

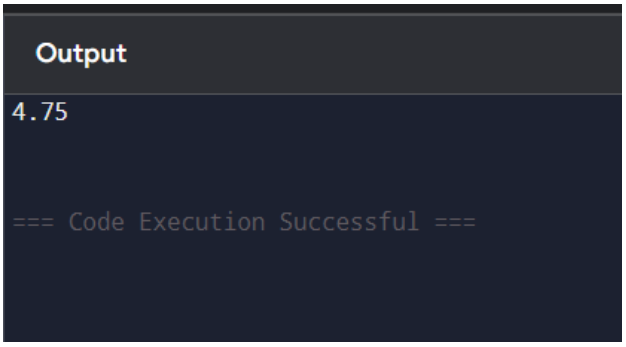
```
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    float num = 5.75;

    Test t = num;

    t.display();

    return 0;
}
```



12. Write a program to convert UDT to basic data type (float)

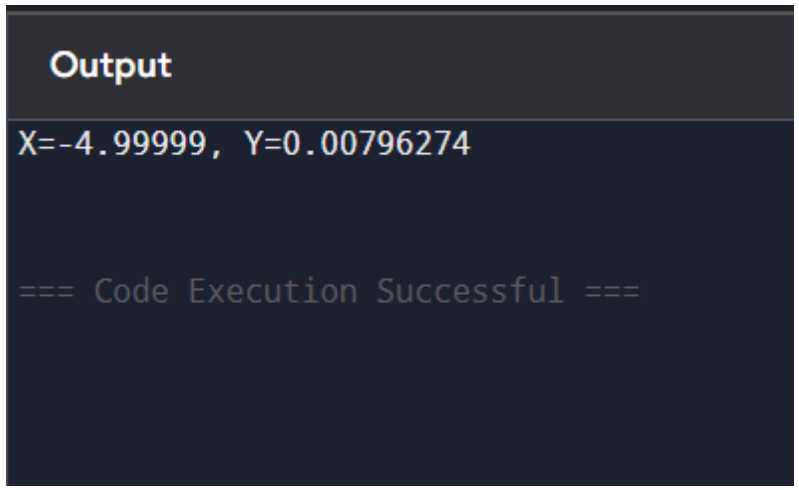
```
class Test {
public:
    operator data_type() {
        // conversion code
    }
};
```

```
#include <iostream>
using namespace std;

class Convert {
    float v;

public:
    Convert(float x = 0) {
        v = x;
    }

    operator float() {
        return v;
    }
};

int main() {
    Convert c(4.75);
    float x = c;
    cout << x << endl;
```



```
    return 0;
}
```

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system.

Polar p(10,5);
Cartesian c = p;
c.show();

```
#include <iostream>
#include <cmath>
using namespace std;

class PolarCoord {
    float radius, angle;

public:
    PolarCoord(float r = 0, float a = 0) {
        radius = r;
        angle = a;
    }

    float getRadius() { return radius; }
    float getAngle() { return angle; }
};
class RectCoord {
    float x, y;
public:
    RectCoord(PolarCoord p) {
        x = p.getRadius() * cos(p.getAngle());
        y = p.getRadius() * sin(p.getAngle());
    }

    void display() {
        cout << "X=" << x << ", Y=" << y << endl;
    }
};

int main() {
    PolarCoord p(5, 3.14);
    RectCoord r = p;
    r.display();
    return 0;
}
```